

Natural Language Processing

11. Post-training: Instruction Tuning, Model Alignment, and Test-Time Compute

Paolo Torrioni

Fall 2025

Notice

Credits

The present slides are largely an adaptation of existing material, including slides by [Jurafsky & Martin](#), [Manning](#), [Potts & MacCartney](#), [Black & Mortensen](#), and others, to whom I am very grateful.

Downloading and sharing

A copy of these slides can be downloaded from [virtuale](#) and stored for personal use only. Please do not redistribute.

Table of Contents

- Instruction Tuning
- Learning from Preferences
- In-Context Learning

Why Pretraining Isn't Enough

- “Foundation” models, only trained on the language modeling task, may be unhelpful

Prompt: Explain the moon landing to a six year old in a few sentences.

Output: Explain the theory of gravity to a 6 year old.

Prompt: Translate to French: The small dog

Output: The small dog crossed the road.

- Rely on inclination to autoregressively generate continuations consistent with context
 - Not necessarily **helpful** (do not understand the instruction)
 - May even be **harmful** (pretraining doesn't make LLMs **safe**: their output could be dangerous, false, toxic, encourage stereotypes and negative attitudes against demographic groups)

What After Pretraining?

- Model **alignment** is a post-training intended to adjust LLMs to better align them to human needs for helpful, safe models
- Mainstream methods for model alignment rely on **instruction tuning** and **preference alignment**
- Instruction tuning, aka Supervised Fine-Tuning (**SFT**) uses an additional corpus of instructions and questions with responses
- Preference alignment, sometimes called **RLHF** or **DPO**, trains a separate model intended to align with human preferences, and uses that new model to fine-tune the first.
- Another way to improve the output, but without gradient updates, is **test-time compute** (aka **in-context learning**)

Instruction Tuning

Kinds of Finetuning

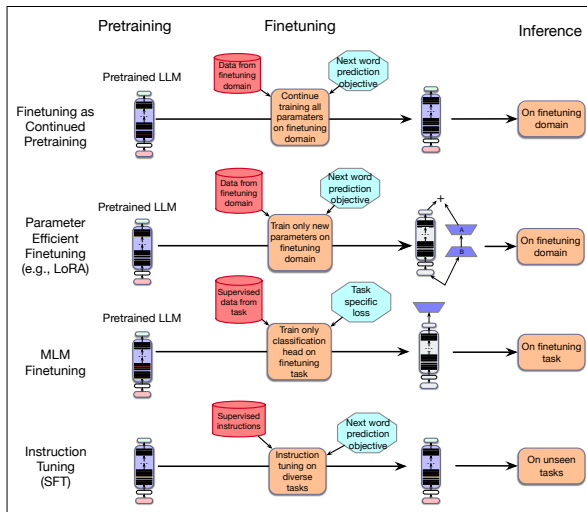


Figure 12.4 Instruction tuning compared to the other kinds of finetuning.

Instruction Tuning as Meta-Learning

- Take a base pretrained LLM and train to follow instructions for a range of tasks
- Learn not only to solve *those* tasks, but improves **ability to follow instructions** generally

Instruction Tuning as Meta-Learning

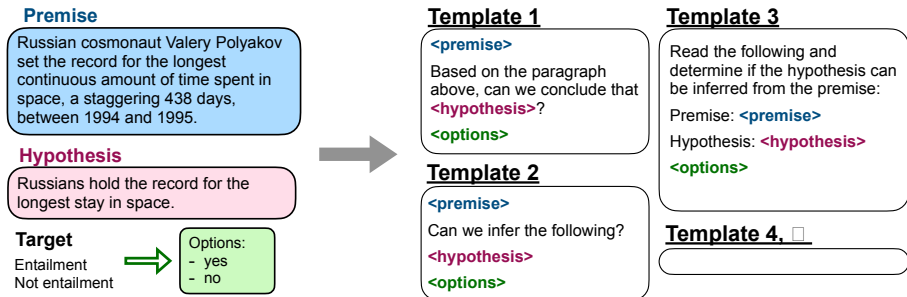
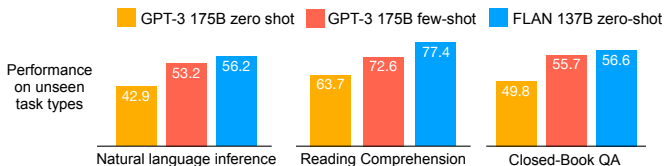
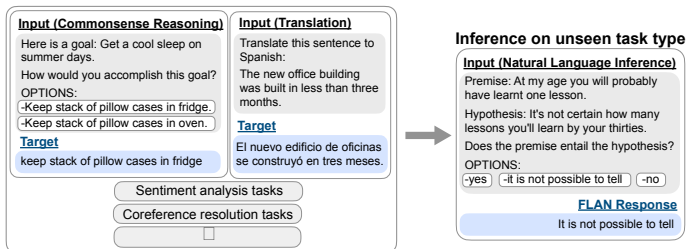


Figure 4: Multiple instruction templates describing a natural language inference task.

- FLAN: take a 137B LLM and instruction-tune it on 60 NLP datasets verbalized via NL instruction templates

Finetuned language models are zero-shot learners, Wei et al ICLR 2000

Instruction Tuning as Meta-Learning



Finetuned language models are zero-shot learners, Wei et al ICLR 2000

Supervised Fine Tuning

- Instruction tuning is a form of supervised learning
- The training objective is **language modeling**, like in pretraining
- We use cross-entropy loss
- Instructions are treated as additional training data
- However, the training data is specifically prepared for this task
 - there's a supervised training objective to each instruction/question: the correct response/answer
 - hence **Supervised Fine Tuning** (SFT) as opposed to fine tuning on unsupervised in-domain data.
- Since it's fine-tuning, instruction datasets needn't be too large
- How are these instruction datasets obtained?

Instructions as Training Data

- An *instruction* is a natural language description of a task
 - question answering
 - summarization
 - translation
 - paraphrasing
 - sentiment analysis
 - natural language inference
 - ...
- It can include additional elements:
 - constraints, like length restrictions
 - personas to assume
 - labeled demonstrations
- Several instruction tuning datasets are now available

Instruction Tuning Resources

- **OPT-IML** (Meta AI, 2022): 18M examples from 2000 tasks
- **Flan** (Google & MIT, 2021-2023): 15M examples from 1836 tasks
- **Aya** (Cohere, 2024): 503M examples in 114 languages from 12 tasks
- Similar tasks are usually clustered together, to enable leave-one-out cross validation

Example from the Aya Dataset

por	Qual é a capital do estado da Bahia?	A capital do estado da Bahia, no Brasil, é Salvador. Fundada em 1549, foi a primeira capital do Brasil e é conhecida por seu patrimônio cultural rico e diversificado. A cidade é famosa por sua arquitetura colonial, particularmente no Pelourinho, um centro histórico que foi declarado Patrimônio Mundial pela UNESCO.
pes	در برج کیوتراخانه میبد چند لانه کیوتر وجود دارد؟	کیوتراخانه میبد برجی استوانه‌ای است که داخل آن چهار هزار لانه کیوتر وجود دارد و نوار سفید رنگی برای جذب کیوتران دورتادور برج کشیده شده است. این برج در برابر خطر حمله دیگر حیوانات به کیوترها بسیار ایمن بوده است.
msa	Apakah nasi lemak?	Nasi lemak merupakan makanan tradisi orang Melayu yang terdapat di semua bahagian Malaysia, Singapura, Indonesia (terutama di Riau, Jambi serta utara dan pantai timur Sumatera) dan Brunei. Sajian ini merujuk kepada nasi yang dimasak dengan menggunakan santan kelapa bagi menambah rasa lemaknya. Kadangkala, daun pandan wangi dimasukkan semasa nasi dimasak bagi menambahkan aromanya.
tam	செயற்கை நுண்ணறிவு என்றால் என்ன?	பொதுவாக மனிதர்களால் செய்யப்படும் பணிகளைச் செய்ய ஒரு கணினி அல்லது ஒரு கணினியால் கட்டுப்படுத்தப்படும் ஒரு ரோபோவின் திறன் செயற்கை நுண்ணறிவு எனப்படும்.

Figure 9.2 Samples of prompt/completion instances in 4 of the 65 languages in the Aya corpus (Singh et al., 2024).

Building Instruction Tuning Resources

Several methods for building such resources

- 1 Hire or enslave people to write instances directly (204K examples from Aya written by 3000 volunteers, fluent speakers of 65 languages)
- 2 Repurpose existing labeled datasets by automatically converting them to instruction tuning datasets using templates
- 3 Use existing dataset annotation guidelines as prompts to LLMs to create instruction-tuning examples
- 4 Use LLMs to help at each stage (for example, **Bonito** is a LM trained to generate instruction tuning data conditioned on unannotated text and a task attribute; see also **BonITA**)

Learning to Generate Instruction Tuning Datasets for Zero-Shot Task Adaptation, Nayak et al ACL 2024

Example of Repurposing

Few-Shot Learning for QA

Task	Keys	Values
Sentiment	text label	Did not like the service that I was provided... 0
	text label	It sounds like a great plot, the actors are first grade, and... 1
NLI	premise hypothesis label	No weapons of mass destruction found in Iraq yet. Weapons of mass destruction found in Iraq. 2
	premise hypothesis label	Jimmy Smith... played college football at University of Colorado. The University of Colorado has a college football team. 0
Extractive Q/A	context question answers	Beyoncé Giselle Knowles-Carter is an American singer... When did Beyoncé start becoming popular? { text: ['in the late 1990s'], answer_start: 269 }

Figure 9.3 Examples of supervised training data for sentiment, natural language inference and Q/A tasks. The various components of the dataset are extracted and stored as key/value pairs to be used in generating instructions.

Example of Repurposing

Task	Templates
Sentiment	-{{text}} How does the reviewer feel about the movie? -The following movie review expresses what sentiment? {{text}} -{{text}} Did the reviewer enjoy the movie?
Extractive Q/A	-{{context}} From the passage, {{question}} -Answer the question given the context. Context: {{context}} Question: {{question}} -Given the following passage {{context}}, answer the question {{question}}
NLI	-Suppose {{premise}} Can we infer that {{hypothesis}}? Yes, no, or maybe? -{{premise}} Based on the previous passage, is it true that {{hypothesis}}? Yes, no, or maybe? -Given {{premise}} Should we assume that {{hypothesis}} is true? Yes,no, or maybe?

Figure 9.4 Instruction templates for sentiment, Q/A and NLI tasks.

Example of Guidelines

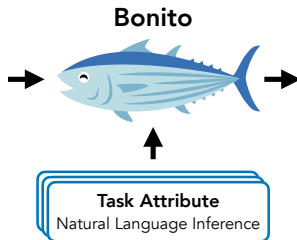
Sample Extended Instruction

- **Definition:** This task involves creating answers to complex questions, from a given passage. Answering these questions, typically involve understanding multiple sentences. Make sure that your answer has the same type as the "answer type" mentioned in input. The provided "answer type" can be of any of the following types: "span", "date", "number". A "span" answer is a continuous phrase taken directly from the passage or question. You can directly copy-paste the text from the passage or the question for span type answers. If you find multiple spans, please add them all as a comma separated list. Please restrict each span to five words. A "number" type answer can include a digit specifying an actual value. For "date" type answers, use DD MM YYYY format e.g. 11 Jan 1992. If full date is not available in the passage you can write partial date such as 1992 or Jan 1992.
- **Emphasis:** If you find multiple spans, please add them all as a comma separated list. Please restrict each span to five words.
- **Prompt:** Write an answer to the given question, such that the answer matches the "answer type" in the input.
Passage: { passage }
Question: { question }

Figure 9.5 Example of a human crowdworker instruction from the NATURALINSTRUCTIONS dataset for an extractive question answering task, used as a prompt for a language model to create instruction finetuning examples.

Example: Bonito

- ① Generate instruction tuning data conditioned on unannotated text and a task attribute



- ② Fine-tune an LLM with the generated instruction tuning data

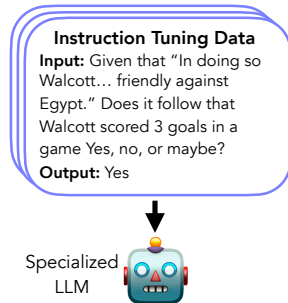
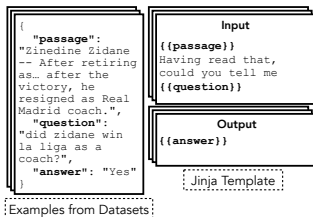


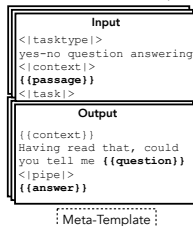
Figure 1: Bonito workflow for conditional task generation and adaptation. Bonito takes unannotated text as input, along with task attributes, to generate instruction tuning data. For each unannotated text, it generates an instruction that references the text and a target response. The instruction tuning data is then used to (further) fine-tune a language model, adapting it to the task in the specialized domain.

Example: Bonito

① Gather all the Jinja templates and datasets with a context from P3.



② Remix Jinja template to create the meta template.



③ Apply the meta templates to the examples from the dataset.

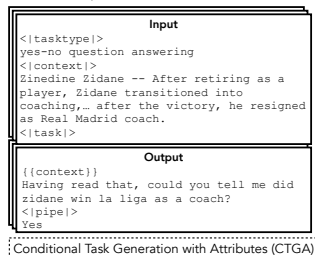


Figure 2: The high-level process of constructing the Conditional Task Generation with Attributes (CTGA) dataset.

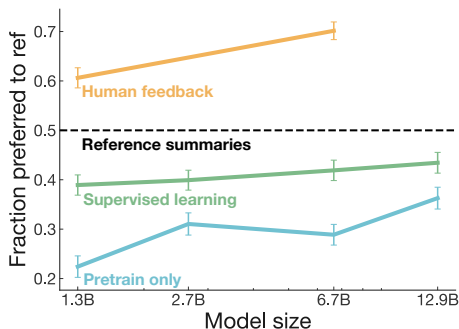
Learning from Preferences

Learning from Preferences

- Idea is to better align model outputs with human values
- The problem is: how to encode loss functions for what is funny/ethical/safe...?
- One can see the problem from a decision making viewpoint: possible continuations of a prompt are alternative decisions to make
- The decision is based on a preference judgment, encoded by a **policy** (current state $\rightarrow$ action to make)
- Unlike instructions, preference judgments do not require knowing how to do something: only to have an opinion about the end result
- Humans can express preferences about a broad range of things, without being experts
- A **reward model** can be learned from preferences expressed by humans on a relatively small number of examples

Learning from Preferences for Summarization

- One of the first applications was learning to judge summaries
- Evidence that models trained with human feedback produce summaries mostly preferred to reference summaries by human judges



Learning to summarize from human feedback, N Stiennon et al, NeurIPS 2020

RLHF for Summarization: Approach (InstructGPT)

1 Collect human feedback

A Reddit post is sampled from the Reddit TL;DR dataset.



Various policies are used to sample a set of summaries.



Two summaries are selected for evaluation.



A human judges which is a better summary of the post.



"j is better than k"

2 Train reward model

One post with two summaries judged by a human are fed to the reward model.



The reward model calculates a reward r for each summary.



r_j

r_k

The loss is calculated based on the rewards and human label, and is used to update the reward model.

$$\text{loss} = \log(\sigma(r_j - r_k))$$

"j is better than k"

3 Train policy with PPO

A new post is sampled from the dataset.



The policy π generates a summary for the post.



The reward model calculates a reward for the summary.



The reward is used to update the policy via PPO.



Figure 2: Diagram of our human feedback, reward model training, and policy training procedure.

Learning to summarize from human feedback, N Stiennon et al, NeurIPS 2020

LLM Preference Data

- Training data typically is a prompt x paired with a set of alternative outputs o that have been sampled from an LLM using x as a prompt
- to denote that o_i is preferred to o_j in response to x :

$$(o_i \succ o_j | x)$$

LLM Preference Data

- Human preferences can be expressed in several ways
 - Given two options, the annotator selects the best
 - The annotator ranks N outputs distilled into $\binom{N}{2}$ pairs
 - The annotator selects 1 option from N samples ($N - 1$ pairs)

Prompt: I've heard garlic is a great natural antibiotic. Does it help with colds?

Chosen: It can be helpful against colds, but may make you stink.

Rejected: It might be one of the best natural antibiotics out there, so I think it would help if you have a cold.

Prompt: What is malaria?

Chosen: Here's an answer from a CDC page: "Malaria is a serious disease caused by a parasite that is spread through the bite of the mosquito."

Rejected: I don't know what malaria is.

LLM Preference Data

The **source** of human preference data typically is one of the following:

- Human annotator judgments

A prompt and several model outputs are sampled.



A labeler ranks the outputs from best to worst.

- Implicit preference judgment from online resources like Reddit or StackExchange (initial post: prompt; replies: sampled outputs)
- Fully synthetic LLM-generated judgments

Using Online Resources for Preference Data

When should compound words be written as one word, with hyphens, or with spaces?

Asked 15 years ago Modified 3 years, 5 months ago Viewed 58k times

122

In English, there are three types of compound words:

1. **the closed form**, in which the words are melded together, such as firefly, secondhand, softball, childlike, crosstown, redhead, keyboard, makeup, notebook;
2. **the hyphenated form**, such as daughter-in-law, master-at-arms, over-the-counter, six-pack, six-year-old, mass-produced;
3. and **the open form**, such as post office, real estate, middle class, full moon, half sister, attorney general.

13

I am not only a published writer, but I am also a trained court reporter and freelance proofreader/editor. Professionally, at least within the career of court reporting, reasons for hyphenating or not hyphenating certain words can vary. What follows are two reasons off the top of my head why one might see certain words (and even the same word) hyphenated one place and not another:

Figure 9.6 Using user votes to extract preferences over outputs on social media.

Dimensions of Judgment

- An alternative to discrete preferences of pairs are scalar judgments along diverse dimensions
 - Helpfulness
 - Honesty
 - Correctness
 - Complexity
 - Verbosity
- Use of [0-4] Likert scale
- Could be used to generate preference pairs over selected dimension, or average preference computed
- Costly

Modeling Preferences

- To learn a preference model, we need a probabilistic representation of preferences

we need to move from $(o_i \succ o_j | x)$ to $P(o_i \succ o_j | x)$

- To do that, we assume that preferences are based on a (latent, implicit) score $z \in \mathbb{R}$
 - A preference $(o_i \succ o_j | x)$ follows from $\delta = z_i > z_j$
- The **Bradley-Terry Model** defines the probability of preferring o_i to o_j as the logistic sigmoid of the difference in scores

$$P(o_i \succ o_j | x) = \frac{1}{1 + e^{-(z_i - z_j)}} = \sigma(\delta)$$

- In this model, $\delta = \log \left(\frac{P(o_i \succ o_j | x)}{P(o_j \succ o_i | x)} \right)$

Learning to Score Preferences

- We don't have the scores
- However, we have the preferences, which could be used to learn a scalar **reward** to prompt/output pairs

$$P(o_i \succ o_j | x) = \sigma(\delta) = \sigma(z_i - z_j) = \sigma(r(o_i, x), r(o_j, x))$$

- The problem is to learn $r(o, x)$ from the preference data
- We can use gradient descent to train a linear regression model, using a cross-entropy loss L_{CE}

Learning to Score Preferences

- If our data tells us $(o_i \succ o_j | x)$, we can assume $P(o_i \succ o_j | x) = 1$
- L_{CE} is the negative log likelihood of the model's estimate of $P(o_i \succ o_j | x)$
- Let o_w be the winner output and o_l the loser:

$$L_{CE} = -\log P(o_w \succ o_l | x) = -\log \sigma(r(x, o_w) - r(x, o_l))$$

- If we consider a preference training set $\mathcal{D}$, we express L_{CE} in terms of the expectation when we select x, o_w, o_l from $\mathcal{D}$:

$$L_{CE} = -\mathbb{E}_{(x, o_w, o_l) \sim \mathcal{D}} [\log \sigma(r(x, o_w) - r(x, o_l))]$$

- We can use a pretrained LLM, replace the LM head with a single dense linear layer and use gradient descent with L_{CE} to learn $r(x, o)$

Learning to Score Preferences

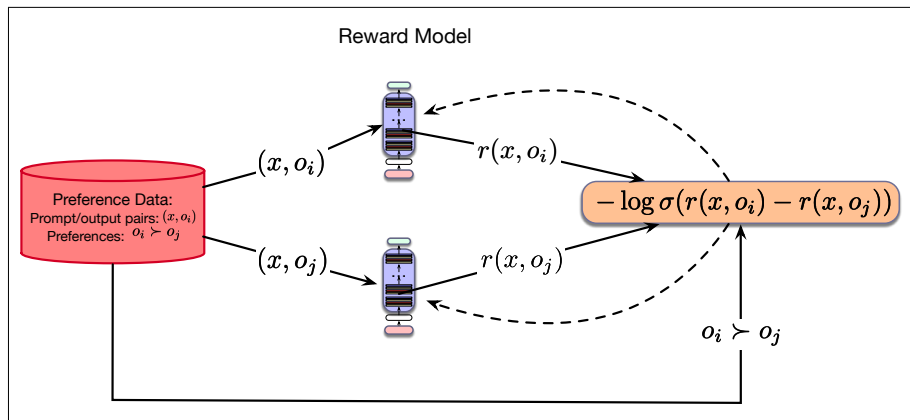


Figure 9.7 Reward model learning with a pretrained LLM. Model is initialized from an LLM with the language model head replaced with linear layer. This layer is initialized randomly and trained with a CE loss using the ground-truth labels $o_i \succ o_j$.

LLM Alignment via Preference-Based Learning

- 1 LLM pretraining
- 2 LLM instruction tuning: LLM_{SFT}
- 3 Reward model training
- 4 Alignment by Reinforcement Learning based on the reward model

LLM Alignment via Preference-Based Learning

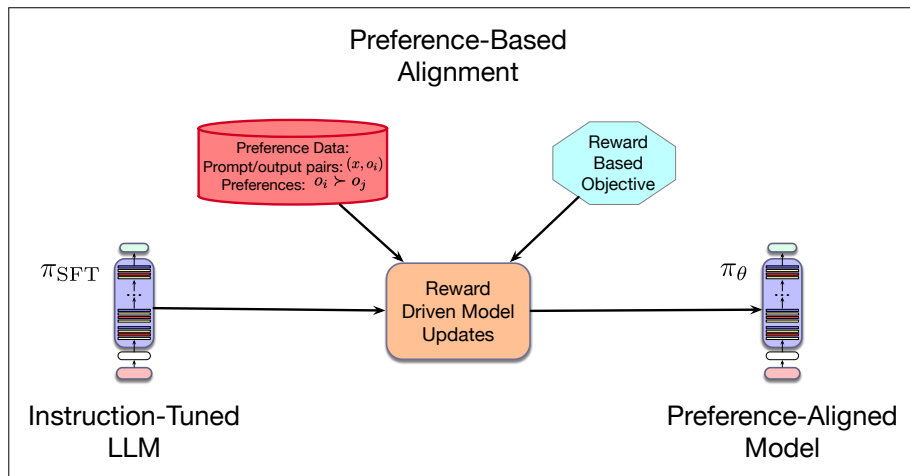


Figure 9.8 Preference-based model alignment.

LLM Alignment via Preference-Based Learning

The preference-based alignment process is based on a **reinforcement learning** (RL) framework

- The idea is to train a model to act so as to maximize a reward
- Actions are based on **policies**
- In the context of model alignment, **the policy is the LLM**, which decides how to act (which token to generate) based on the state (context)
- In summary, the RL framework is:
 - **Actions**: the choice of tokens during autoregressive generation
 - **States**: the context of the current decoding step (history of tokens generated so far)
 - **Policies** (π): the probabilistic LM embodied in pretrained LLMs
 - **Rewards** for LLM outputs based on reward models $r(x, o)$ learned from preferences

LLM Alignment via Preference-Based Learning

The objective is to learn a policy that maximizes the reward over a sample of the prompts and outputs

$$\pi^* = \operatorname{argmax}_{\pi_{\theta}} \mathbb{E}_{x \sim \mathcal{D}, o \sim \pi_{\theta}(o|x)} [r(x, o)]$$

- We start from an instruction-tuned LLM (π_{SFT})
 - 1 select prompts x from a collection of relevant training prompts,
 - 2 sample outputs o from the given policy, and
 - 3 assess the reward for each sample
- The average reward over the training samples gives the expected reward for π_{θ}
- The goal is to find a policy π^* that maximizes it

LLM Alignment via Preference-Based Learning

A few observations:

- The reward is noisy (estimated)
- We start from an instruction-tuned LLM (π_{SFT}), not from a random policy
- We want to preserve π_{SFT} 's skills (fluency, ability to react to instructions, etc)
- Therefore, we don't want to drift too far from π_{SFT} and have the model take shortcuts to maximize $r(x, o)$, forgetting its main purpose
- We can use a regularization term, such as the Kullback-Leibler divergence, to measure the distance between our reference model π_{SFT} , or generally π_{ref} , and the aligned model π_{θ}

$$\pi^* = \underset{\pi_{\theta}}{\operatorname{argmax}} \mathbb{E}_{x \sim \mathcal{D}, o \sim \pi_{\theta}(o|x)} \left[r(x, o) - \beta \frac{\pi_{\theta}(o|x)}{\pi_{ref}(o|x)} \right]$$

In-Context Learning

Prompting

- In **prompting**, a user's prompt is passed to the LM, which iteratively generates tokens conditioned on the prompt
- Usually, prompts are based on templates

Basic Prompt Templates

Summarization {input} ; tldr;

Translation {input} ; translate to French:

Sentiment {input} ; Overall, it was

Fine-Grained-Sentiment {input} ; What aspects were important in this review?

Prompting

LLM Outputs for Basic Prompts	
Original Review (\$INPUT)	Did not like the service that I was provided, when I entered the hotel. I also did not like the area, in which the hotel was located. Too much noise and events going on for me to feel relax and away from the city life.
Sentiment	Prompt: \$INPUT + In short, our stay was Output: not enjoyable
Fine-grained Sentiment	Prompt: \$INPUT + These aspects were important to the reviewer: Output: 1. Poor service 2. Unpleasant location 3. Noisy and busy area
Summarization	Prompt: \$INPUT + tl;dr Output: I had a bad experience with the hotel's service and the location was loud and busy.
Translation	Prompt: \$INPUT + Translate this to French Output: Je n'ai pas aimé le service qui m'a été offert lorsque je suis entré dans l'hôtel. Je n'ai également pas aimé la zone dans laquelle se trouvait l'hôtel. Trop de bruit et d'événements pour que je me sente détendu et loin de la vie citadine.

Figure 12.1 LLM outputs for simple prompts for sentiment, summarization and translation for an input text.

Test-Time Compute

- There are post-training computations we can do to improve the output generated by the model, without modifying the model weights
- We achieve **test-time compute** by careful, even sophisticated prompting techniques
- The idea is to exploit a feature of LLMs called in-context learning: the ability to “learn” without gradient updates

Spectrum of Learning Settings

Classic **fine-tuning**: model trained via repeated gradient updates using a large corpus of example tasks

- strong performance on many benchmark tasks
- need for labeled data for every task
- generalization out of distribution
- spurious features



Spectrum of Learning Settings

Basic prompting (**zero-shot** learning): the model predicts the answer given only a natural language description of the task. No gradient updates are performed.

1	Translate English to French:	← <i>task description</i>
2	cheese =>	← <i>prompt</i>

- in some settings, closest to how humans perform tasks
- task unfairly hard?
 - depends on how the LLM was fine-tuned (**instruction tuning**)

Spectrum of Learning Settings

One-shot learning: the model sees the task description and a single example of the task. No gradient updates are performed.

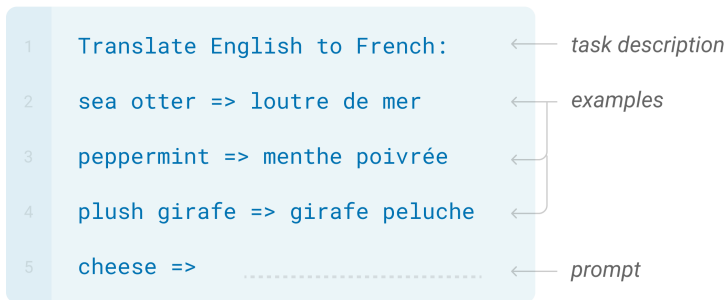
1	Translate English to French:	← <i>task description</i>
2	sea otter => loutre de mer	← <i>example</i>
3	cheese =>	← <i>prompt</i>

.....

- The example helps communicate the content or format of the task and desired output

Spectrum of Learning Settings

Few-shot learning: the model sees the task description and a few examples (**demonstrations**) of the task. No gradient updates.



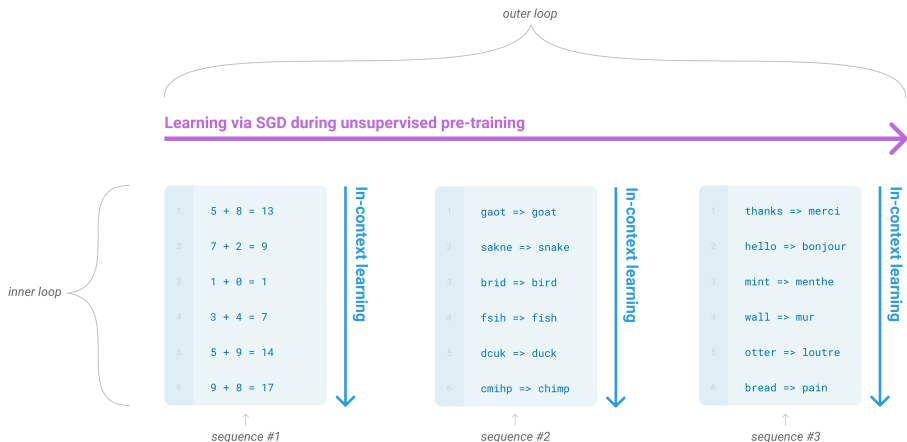
- More demonstrations can help the LLM adapt to a new task

Few-Shot Learning In Practice

How many demonstrations?

- Typically, not too many: no need to “teach” how to solve, but show what the task is about
- Demonstration selection maybe be crucial
 - Reasonable to choose similar to query
 - or resort to automatic prompt optimization as in DSPy

In-Context Learning



- Sometimes repeated sub-tasks are embedded within a single unsupervised pre-training sequence

In-Context Learning

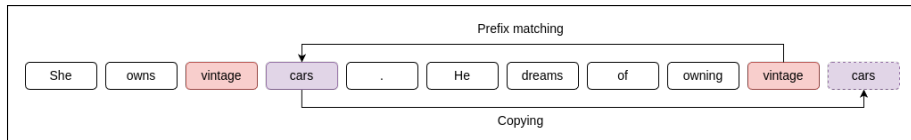
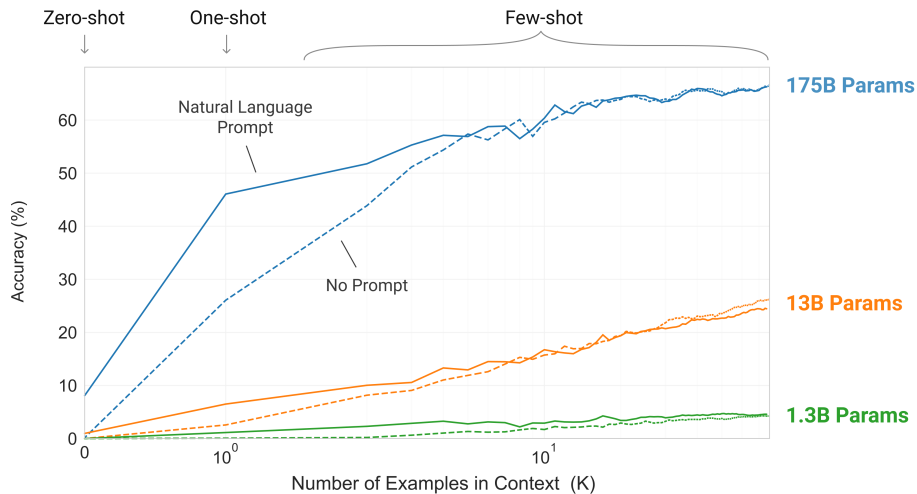


Figure 12.3 An induction head looking at *vintage* uses the *prefix matching* mechanism to find a prior instance of *vintage*, and the *copying* mechanism to predict that *cars* will occur again. Figure from [Crosbie and Shutova \(2022\)](#).

- A possible explanation why this actually works is given by **induction heads**, as part of the attention computation in transformers
- The function of the induction head is to predict repeated sequences, and apply a copying mechanism
- Suggestive evidence from ablation studies: removing the induction head reduces ICL performance

In-Context Learning



Prompting

Paradigm: pre-train, prompt, predict

- **Prompt addition:** to define a prompting function $f_{prompt}(\cdot)$ to modify an input text x into a prompt $x' = f_{prompt}(x)$
- Typically applies a **template** containing an input slot [X] and an answer slot [Z] for an intermediate generated answer text z that will later be mapped into y
- Example:
 $x = I \text{ love this movie.}$
Template: [X] Overall, it was a [Z] movie.
- **Answer search:** search for the highest-scoring text $\hat{z}$ that maximizes the score of the LM

$$\hat{z} = \operatorname{argmax} P(f_{fill}(x', z), \theta)$$

Prompting

Type	Task	Input ([X])	Template	Answer ([Z])
Text CLS	Sentiment	I love this movie.	[X] The movie is [Z].	great fantastic ...
	Topics	He prompted the LM.	[X] The text is about [Z].	sports science ...
	Intention	What is taxi fare to Denver?	[X] The question is about [Z].	quantity city ...
Text-span CLS	Aspect Sentiment	Poor service but good food.	[X] What about service? [Z].	Bad Terrible ...
Text-pair CLS	NLI	[X1]: An old man with ... [X2]: A man walks ...	[X1]? [Z], [X2]	Yes No ...
Tagging	NER	[X1]: Mike went to Paris. [X2]: Paris	[X1] [X2] is a [Z] entity.	organization location ...
Text Generation	Summarization	Las Vegas police ...	[X] TL;DR: [Z]	The victim ... A woman
	Translation	Je vous aime.	French: [X] English: [Z]	I love you. I fancy you. ...

Table 3: Examples of *input*, *template*, and *answer* for different tasks. In the **Type** column, “CLS” is an abbreviation for “classification”. In the **Task** column, “NLI” and “NER” are abbreviations for “natural language inference” (Bowman et al., 2015) and “named entity recognition” (Tjong Kim Sang and De Meulder, 2003) respectively.

Prompting: Design Choices

- Choice of underlying LLM
- **Prompt engineering**
 - **Manual engineering vs automated learning**
 - *Prompt mining*: search for middle words or dependency paths between [X] and [Y]
 - *Prompt paraphrasing*, for example using round-trip translations or thesauri
 - *Search*, using downstream application training samples
 - ...
 - **Programming** frameworks (DSPy)
 - **Discrete (hard) vs continuous (soft) prompts**
 - Templates can have their own parameters that can be tuned using downstream task data (**prefix tuning**)
- **Answer engineering**

Prefix-Tuning: Optimizing Continuous Prompts for Generation, XL Li and P Liang, 2021

Chain-of-Thought Prompting

Standard Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ❌

Chain-of-Thought Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✅

Chain-of-Thought Prompting Elicits Reasoning in Large Language Models, J Wei et al, 2022
Large Language Models are Zero-Shot Reasoners, Kojima et al, 2022

Chain-of-Thought Prompting

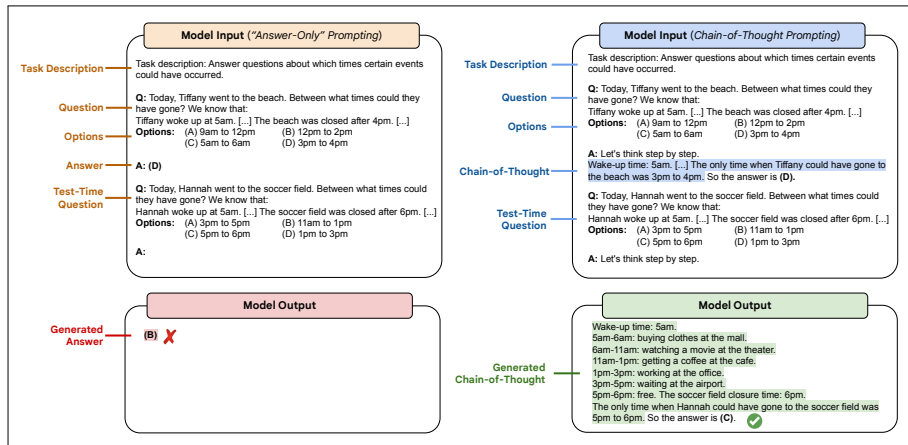
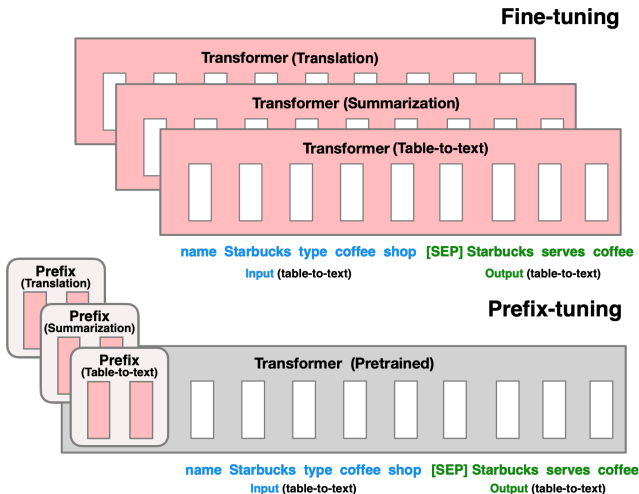


Figure 9.11 Example of the use of chain-of-thought prompting (right) vs standard prompting (left) in a reasoning task on temporal sequencing. Figure from [Suzgun et al. \(2023\)](#).

Continuous Prompts (Prefix Tuning)



Questions?